

C1 - Automatiser l'extraction des données

Projet : référentiel automatisé des pays

Finalité du document

Présenter les choix, l'architecture, les scripts et les preuves permettant de démontrer l'extraction automatisée depuis cinq familles de sources.

Élément	Valeur
Candidat	Abderrauof Metidji
Compétence	C1 - Automatiser l'extraction des données
Version	1.0 - document de travail
Dépôt	rncp-e1-countries-pipeline
Statut	Scripts livrés ; preuves réelles à compléter après exécution complète

1. Contexte et objectif

L'objectif est de constituer automatiquement un référentiel enrichi des pays. Les cinq sources contiennent des données de même nature et convergent vers une représentation commune identifiée par les codes ISO alpha-2 et alpha-3. Ce choix limite les difficultés métier et permet de concentrer la démonstration sur l'extraction, la robustesse, la traçabilité et la reproductibilité.

Périmètre C1

C1 couvre l'extraction et la sauvegarde des données brutes. Les règles d'harmonisation complètes seront détaillées en C3, la modélisation relationnelle en C4 et l'exposition par API en C5.

2. Architecture technique

Le pipeline est lancé par une commande unique. Chaque extracteur est isolé dans un module Python et produit une sortie horodatée dans une zone RAW. Un manifeste d'exécution centralise le statut, le nombre de lignes, la taille du fichier et son empreinte SHA-256.

Source	Entrée	Traitement	Sortie RAW
API Web	REST Countries, JSON	HTTP, contrôles, tentatives, sélection des champs	JSON
Web scraping	Tableau HTML public	Téléchargement, copie HTML, parsing du tableau	HTML + JSON
Fichier	Référentiel ISO	Contrôle schéma, valeurs obligatoires, doublons	CSV
Base de données	PostgreSQL	Requête paramétrée, sélection des colonnes	Parquet
Big Data	CSV converti en Parquet	Interrogation SQL avec DuckDB, projection et filtre	Parquet

Flux logique : Sources → extracteurs spécialisés → zone RAW → manifeste d'exécution → traitements ultérieurs de C2 à C5.

3. Description des cinq extractions

3.1 API Web

L'extracteur appelle REST Countries et demande uniquement les champs nécessaires. Il contrôle le statut HTTP, impose un délai maximal, applique trois tentatives avec attente exponentielle en cas de timeout ou d'erreur de connexion, vérifie que la réponse est une liste puis sauvegarde un JSON normalisé.

Champs retenus : iso2, iso3, nom courant, nom officiel, capitale, région, sous-région, population, superficie, monnaies et langues.

[PREUVE À INSÉRER] Capture de la documentation de l'endpoint et d'une réponse JSON réelle.

3.2 Web scraping

Le script télécharge une page Wikipédia contenant un tableau démographique. La page HTML complète est conservée afin de garantir la traçabilité. BeautifulSoup localise le tableau et pandas.read_html transforme les lignes HTML en données structurées. Le script contrôle la présence des colonnes pays et population et nettoie les références entre crochets.

[PREUVE À INSÉRER] Capture de la page, du tableau et de l'inspecteur HTML montrant la balise table.

3.3 Fichier CSV

Le fichier ISO est lu en UTF-8. Le script vérifie la présence des colonnes iso2, iso3 et country_name, refuse les valeurs obligatoires manquantes et supprime les doublons sur les codes ISO. La copie RAW reste indépendante du fichier d'entrée.

Résultat du test local livré : 10 lignes valides extraites depuis le fichier d'exemple.

3.4 Base PostgreSQL

PostgreSQL est lancé avec Docker Compose et initialisé par database/init.sql. SQLAlchemy ouvre la connexion et exécute une requête paramétrée filtrant les pays actifs. Seules les colonnes nécessaires sont sélectionnées. La connexion et le pool sont fermés dans tous les cas. Les erreurs SQL sont journalisées et remontées comme échec d'extraction.

[PREUVE À INSÉRER] Capture du conteneur actif, de la requête et du fichier Parquet produit.

3.5 Parquet et DuckDB

Le jeu d'indicateurs est converti en Parquet avec compression ZSTD, puis interrogé directement par DuckDB. La requête projette uniquement six colonnes, filtre les codes et populations non renseignés, trie le résultat et applique une limite configurable. Cette approche évite un chargement intégral inutile en mémoire et démontre l'usage d'un moteur analytique adapté aux fichiers colonnes.

Résultat du test local livré : 10 lignes extraites et fichier Parquet généré avec succès.

4. Automatisation, robustesse et traçabilité

Mécanisme	Mise en œuvre
Commande unique	<code>python -m src.main --sources all --continue-on-error</code>
Configuration	Fichier YAML centralisant URLs, chemins, délais et limites
Erreurs réseau	Timeout, contrôle HTTP et trois tentatives exponentielles
Qualité fichier	Contrôle du schéma, valeurs obligatoires et doublons
Journalisation	Logs console et fichier horodaté
Traçabilité	Manifeste JSON avec lignes, octets, SHA-256 et statut
Secrets	URL PostgreSQL surchargeable par variable <code>DATABASE_URL</code>
Reproductibilité	Dépendances figées, Docker Compose et scripts SQL
Tests	pytest ; API simulée pour ne pas dépendre du réseau

5. Structure du dépôt

- `config/settings.yaml` : configuration fonctionnelle et technique
- `src/main.py` : orchestration des cinq extracteurs
- `src/extract/*.py` : un module par famille de source
- `src/common.py` : configuration, logs, écritures atomiques et empreintes
- `database/init.sql` : création et alimentation de la source PostgreSQL
- `data/input` : petits jeux d'exemple reproductibles
- `data/raw` : résultats bruts par source
- `tests` : tests automatisés
- `docs` : guide des preuves et script oral

6. Installation et exécution

6.1 Installation

```
python -m venv .venv
```

```
source .venv/bin/activate # Linux/macOS
```

```
.venv\Scripts\Activate.ps1 # Windows PowerShell
```

```
pip install -r requirements.txt
```

```
docker compose up -d
```

6.2 Exécution

```
python -m src.main --sources all --continue-on-error
```

```
python -m src.main --sources file bigdata
```

```
pytest -q
```

Attention

Les volumes et captures définitifs doivent provenir d'une exécution réelle sur la machine présentée au jury. Aucune valeur de production ne doit être inventée.

7. Résultats et preuves à compléter

Preuve	Attendu	Statut
P1	Réponse JSON réelle de l'API	À capturer
P2	Page HTML et balise table	À capturer
P3	Contrôle du CSV et fichier RAW	Script testé
P4	PostgreSQL actif et extraction réussie	À exécuter
P5	Requête DuckDB et Parquet produit	Script testé
P6	Logs des cinq extractions	À capturer
P7	pytest sans erreur	3 tests passés localement
P8	Historique Git et commits séparés	À constituer

8. Limites et améliorations

- Une page HTML externe peut changer de structure. Le contrôle explicite des colonnes provoque alors un échec visible plutôt qu'un résultat silencieusement incorrect.
- Les indicateurs d'exemple sont volontairement réduits. Pour la démonstration finale, un fichier Parquet public plus volumineux peut remplacer le fichier fourni sans modifier le principe de requêtage.
- Le pipeline C1 conserve les sorties par source. La déduplication et la fusion inter-sources seront traitées dans C3.
- Les secrets ne doivent jamais être écrits dans Git. Le mot de passe du Docker Compose local est uniquement une valeur pédagogique et doit être remplacé dans un environnement partagé.

9. Conclusion

Le projet démontre une extraction automatisée depuis cinq familles de sources au moyen de scripts Python séparés, configurables et testables. Les mécanismes de contrôle, journalisation, reprise réseau, écriture atomique et empreinte de fichier rendent l'exécution compréhensible et traçable. La compétence sera considérée prête lorsque les cinq extractions auront été exécutées avec succès, versionnées et associées à leurs preuves visuelles réelles.

Annexe - Checklist C1

Contrôle A	Contrôle B
☐ API réelle exécutée	☐ Page HTML réellement parsée
☐ CSV contrôlé	☐ PostgreSQL réellement interrogé
☐ Parquet réellement interrogé avec DuckDB	☐ Sorties RAW horodatées
☐ Logs conservés	☐ Manifeste généré
☐ Tests réussis	☐ Dépôt Git individuel
☐ README exécutable	☐ Captures légendées et sans secrets